

Università degli Studi di Napoli
Federico II

A.A. 2026-2027

Tecnologie Web

Indice

0 Parte I --- DNS 3

1. Domain Name System 3
   - Definizione, nomi e gerarchia 3
   - Resource record e attori 3
   - Ricorsione, iterazione e risposte 3

1 Parte II --- HTTP 4

2. Fondamenti del Web e di HTTP 4
   - Richieste, metodi, intestazioni e risposte 4
   - Statelessness e server HTTP 5
3. Rappresentazioni, cache, HTTPS e versioni HTTP 5
   - Content negotiation e compressione 5
   - Caching: memorizzazione, freschezza e validazione 5
   - HTTPS 5
   - Versioni di HTTP 5

2 Parte III --- HTML 6

4. Fondamenti di HTML 6
   - Elementi, attributi, DOM 6
   - Struttura di base 6
   - Elementi semantici e accessibilità 6
5. Form e interazione 6
   - Controlli e attributi 7
   - Validazione e invio 7

3 Parte IV --- CSS 8

6. Fondamenti di CSS 8
   - Selettori 8
   - Specificità e cascata 8
   - Box model e display 8
   - Flexbox e Grid 8
   - Media query e strategie responsive 9
7. CSS moderno, adattivo e accessibile 9

4 Parte V --- JavaScript 11

8. JavaScript: linguaggio, scope, funzioni e oggetti 11
   - Inclusione, sintassi e operatori 11
   - Variabili, hoisting e scope 11
   - Funzioni, espressioni e closure 11
   - Oggetti, riferimenti, metodi, this 11
9. Ereditarietà prototipale, strutture dati, classi, moduli 12
   - Prototipi 12
   - Array, destructuring, iterabili, Map e Set 12
   - Destructuring 12
   - Map e Set 13
   - Moduli 13

5 Parte VI --- Architetture e Best Practice 14

- Architetture client-server e multi-tier 14
- API REST 14
- Sicurezza Web 14
- Best Practice 14

Capitolo 0

Parte I — DNS

1. Domain Name System

Definizione, nomi e gerarchia

DNS (Domain Name System): database **distribuito e gerarchico** di Internet che associa a un nome informazioni descritte tramite *resource record* (RR). Non associa necessariamente un nome a un computer: un nome può identificare servizi, alias, server di posta o nodi amministrativi. DNS non è una semplice rubrica centralizzata.

Un **nome DNS** è una sequenza di **etichette** separate da punti, lette in senso gerarchico **da destra a sinistra**. In `informatica.dieti.unina.it.` la gerarchia è: radice `.` → `it` → `unina` → `dieti` → `informatica`. La radice corrisponde all'etichetta vuota finale; i nomi sono **case-insensitive**. Un nome completo terminato dal punto è un *fully qualified domain name* (FQDN). Uno stesso host può possedere più nomi.

Dominio: nodo dell'albero dei nomi insieme a tutto il suo sottoalbero. **Zona**: porzione del namespace di cui uno o più name server pubblicano autoritativamente i record. **Delega**: trasferimento dell'autorità su un sottoalbero ad altri name server. Se `unina.it` delega `dieti.unina.it`, quest'ultimo resta nel **dominio** `unina.it`, ma non nella **zona** `unina.it`.

Un **name server autoritativo** contiene e distribuisce dati delle zone per cui è responsabile; può servire molte zone e una zona può avere più server autoritativi, anche per ridondanza. Un **resolver ricorsivo** interroga i server necessari fino a ricostruire una risposta, spesso riutilizzando cache.

Resource record e attori

Un record DNS contiene: **owner name** (nome a cui si riferisce), **TTL** (*time to live*, durata in secondi della copia in cache), **class** (di norma `IN`), **type** (natura del record) e **value** (dato).

Attori: l'applicazione o *client* vuole risolvere un nome; lo **stub resolver** del sistema operativo raccoglie la richiesta; il **resolver ricorsivo** (dell'ISP o di un provider) ricerca o recupera dalla propria cache i dati; i **server autoritativi** forniscono record o informazioni sulla delega.

Ricorsione, iterazione e risposte

- **Query ricorsiva**: chi riceve la query deve procurarsi la risposta finale oppure un errore; è il normale rapporto stub → resolver ricorsivo.
- **Query iterativa**: chi risponde dà la migliore informazione disponibile, anche un **referral** (indicazione di server da interrogare in seguito); il resolver segue i referral verso gli autoritativi.

Una richiesta (*question*) contiene principalmente `QNAME`, `QCLASS` e `QTYPE`. Una risposta presenta **header** (identificativo, flag e stato), **question** (domanda ripetuta), **answer** (RR che rispondono), **authority** (autorità o referral) e **additional** (RR ausiliari, ad esempio indirizzi dei name server indicati).

Con `NOERROR` si distinguono **risposta positiva** (RR in `Answer`), **referral** (deleghe in `Authority`) e **NODATA** (nome esistente ma assenza del tipo di record chiesto). **NODATA non equivale a NXDOMAIN**.

Risoluzione a cache vuota (*cold-cache*): per `squids.unina.it` il resolver interroga un server **root**, che indica quelli di `.it`; uno di `.it` indica quelli di `unina.it`; il server di `unina.it` fornisce il record `A` o un'ulteriore delega.

Da ricordare. *Dominio ≠ zona; resolver ricorsivo ≠ autoritativo; query ricorsiva ≠ iterativa; NXDOMAIN ≠ NODATA; TTL DNS ≠ durata della cache HTTP.*

Parte II — HTTP

2. Fondamenti del Web e di HTTP

Internet è la rete globale basata sulla suite TCP/IP; il **World Wide Web** è un sistema di risorse e ipertesti collegati, che usa Internet come infrastruttura. Un **ipertesto** contiene collegamenti (*hyperlink*) ad altri contenuti; **HTML** ne descrive la struttura e **HTTP** la comunicazione.

HTTP (Hypertext Transfer Protocol): protocollo applicativo **client–server**, basato sull'interazione **richiesta–risposta**, che permette di operare su risorse identificate da URL. Nelle versioni 1.1 e 2 usa di norma TCP; HTTP/3 usa QUIC/UDP.

URL (Uniform Resource Locator): riferimento che specifica dove e come raggiungere una risorsa.

Forma generale:

```
scheme://user:password@host:port/path?query#fragment
https://www.example.org:443/corsi/web?q=http#lezione
```

Componenti: **scheme** (es. `https`), **authority/host**, **porta** (se assente quella predefinita), **path**, **query string** (`?chiave=valore`) e **fragment** (`#...`, di norma elaborato dal client e non inviato nella richiesta HTTP).

Richieste, metodi, intestazioni e risposte

Una richiesta HTTP/1.1 contiene **request line** (`METODO target HTTP/versione`), intestazioni, riga vuota e un eventuale **body**; una risposta contiene **status line**, intestazioni, riga vuota e un eventuale body.

Una **intestazione** (*header*) è una coppia nome–valore separata da `:`.

- `Content-Type`: identifica il media type del corpo (`text/html`, `application/json`, …)
- `Content-Length`: indica i byte del corpo quando applicabile
- `Host`: identifica il server virtuale desiderato

Metodo	Semantica essenziale	Safe?	Idempotente?
`GET`	Recupera una rappresentazione.	Sì	Sì
`HEAD`	Come GET, ma senza corpo della risposta.	Sì	Sì
`POST`	Sottopone dati per un'elaborazione/creazione.	No	Non garantito
`PUT`	Crea o sostituisce la rappresentazione della risorsa indicata.	No	Sì
`PATCH`	Applica modifiche parziali.	No	Non garantito
`DELETE`	Elimina la risorsa indicata.	No	Sì

Safe: il client non richiede un cambiamento di stato; **idempotente**: eseguire più volte la stessa operazione ha lo stesso **effetto previsto** che eseguirla una volta.

I codici HTTP sono divisi in cinque famiglie:

- **1xx**: informativi
- **2xx**: successi (200 OK, 201 Created, 204 No Content)
- **3xx**: redirect/gestione cache (301, 302, 304 Not Modified)
- **4xx**: errori lato richiesta (400, 401, 403, 404, 405, 409)
- **5xx**: errori nel trattamento da parte del server (500, 502, 503)

401 significa credenziali mancanti/non valide; **403** significa che l'identità può essere nota ma l'operazione è vietata.

Statelessness e server HTTP

HTTP è stateless: ciascuna richiesta è semanticamente indipendente. Autenticazione persistente, carrello e preferenze richiedono meccanismi applicativi aggiuntivi (cookie, sessioni, token).

Server HTTP può significare:

- Ruolo client–server
- Software (Apache/Nginx)
- Processo
- Macchina

3. Rappresentazioni, cache, HTTPS e versioni HTTP

Content negotiation e compressione

Una risorsa può avere **più rappresentazioni** (lingua, formato, codifica). La **content negotiation** permette al client di inviare preferenze mediante:

- `Accept` (MIME type)
- `Accept-Language`
- `Accept-Encoding`
- `Accept-Charset`

Esempio:

```
Accept: text/html, application/json;q=0.7
Accept-Language: it, en;q=0.8
Accept-Encoding: gzip, br
```

Il server sceglie una rappresentazione disponibile; se nessuna è accettabile può restituire **406 Not Acceptable**. `Content-Language`, `Content-Type` e `Content-Encoding` descrivono la rappresentazione inviata.

La compressione **lossless** (`gzip`, `deflate`, `br`/Brotli) risparmia banda ma richiede CPU.

Caching: memorizzazione, freschezza e validazione

Caching HTTP: conservazione di risposte per riutilizzarle, riducendo latenza, banda, carico server e migliorando la scalabilità.

HTTPS

HTTPS = HTTP + TLS/SSL su TCP (tipicamente porta **443**). TLS fornisce:

- **Cifratura** (riservatezza)
- **Integrità** dei dati
- **Autenticazione** del server (tramite certificati X.509)

Il client verifica che il certificato sia valido e corrisponda al nome del server.

Versioni di HTTP

- **HTTP/1.0**: connessione per ogni richiesta (inefficiente)
- **HTTP/1.1**: connessioni persistenti, pipelining (limitato), header `Host`
- **HTTP/2**: multiplexing su una connessione, compressione header (HPACK), priorità
- **HTTP/3**: multiplexing su QUIC/UDP, migliore gestione delle perdite di pacchetti

Parte III — HTML

4. Fondamenti di HTML

HTML (HyperText Markup Language): linguaggio di markup per descrivere la struttura di una pagina web. Non è un linguaggio di programmazione: non definisce logica, ma **significato** (semantica) e struttura.

Un documento HTML è un albero di **elementi** (nodi) con:

- **Tag di apertura** (`<p>`)
- **Tag di chiusura** (`</p>`)
- **Contenuto testuale**
- **Attributi** (`name="value"`)

Elementi, attributi, DOM

Elementi: possono essere **blocco** (occupano una riga, es. `<div>`, `<p>`) o **inline** (flussi di testo, es. `<span>`, `<a>`).

Attributi globali: `id`, `class`, `lang`, `dir`, `title`, `data-*` (per dati personalizzati).

DOM (Document Object Model): rappresentazione ad albero del documento HTML in memoria. Permette a JavaScript di accedere e modificare la struttura, lo stile e il contenuto della pagina.

Struttura di base

```HTML
<!DOCTYPE html>
<html lang="it">
<head>
  <meta charset="utf-8">
  <title>Titolo della pagina</title>
</head>
<body>
  Contenuto visibile
</body>
</html>
```

Elementi semantici e accessibilità

Elementi semantici: `<header>`, `<footer>`, `<main>`, `<article>`, `<section>`, `<nav>`, `<aside>`, `<figure>`, `<figcaption>`, `<time>`, `<mark>`.

Accessibilità: usare elementi semantici, attributi `alt` per immagini, `aria-*` per componenti complessi, ordini logici e sufficienti contrasti.

Da ricordare. `<div>` e `<span>` sono generici e non semantici; `alt` è obbligatorio per immagini informative; i titoli (`<h1>-<h6>`) non vanno usati per mere dimensioni.

5. Form e interazione

Form: raccolta di controlli per l'input utente, invio a un server. Attributi principali: `action` (URL del server), `method` (`GET` o `POST`).

Controlli e attributi

- `<input type="text">`: testo su una riga
- `<input type="password">`: testo nascosto
- `<input type="email">`: validazione email lato client
- `<input type="number">`: numero
- `<input type="date">`: data
- `<input type="checkbox">`: flag booleano
- `<input type="radio">`: scelta singola tra opzioni
- `<input type="file">`: upload file
- `<input type="submit">`: invio del form
- `<textarea>`: testo su più righe
- `<select>`: menu a tendina
- `<button>`: pulsante generico

Attributi comuni: `name` (identificatore per il server), `value` (valore predefinito), `required`, `disabled`, `readonly`, `placeholder`.

Validazione e invio

Validazione lato client: HTML5 introduce attributi come `required`, `pattern`, `min`, `max`, `minlength`, `maxlength`, `type="email"`.

Invio: può essere via `GET` (parametri in URL) o `POST` (corpo della richiesta).

Da ricordare. `name` è fondamentale per l'invio; `value` è il dato inviato; i radio button con lo stesso `name` sono mutualmente esclusivi.

Parte IV — CSS

6. Fondamenti di CSS

CSS (Cascading Style Sheets): linguaggio per descrivere la presentazione di documenti HTML. Un **ruleset** è formato da **selettore** + **dichiarazione** (`proprietà: valore;`).

Selettori

- Tipo: `p`
- Classe: `.classe`
- ID: `#id`
- Attributo: `[type="submit"]`
- Discendenti: `div p`
- Figli: `div > p`
- Adiacenti: `h1 + p`
- Generici: `h1 ~ p`
- Pseudo-classi: `:hover`, `:active`, `:focus`, `:nth-child()`
- Pseudo-elementi: `::before`, `::after`

Specificità e cascata

Specificità: ordine di priorità tra selettori.

1. Inline (`style=""`)
2. ID
3. Classe, attributo, pseudo-classe
4. Tipo, pseudo-elemento

Cascata: l'ordine dei ruleset conta; in caso di parità di specificità, vince l'ultimo.

Box model e display

Box model: ogni elemento è un rettangolo con:

- **Content**
- **Padding** (spazio interno)
- **Border**
- **Margin** (spazio esterno)

`box-sizing: border-box` include padding e border nella larghezza.

Display:

- `block`: occupa una riga, larghezza 100%
- `inline`: flusso di testo, larghezza auto
- `inline-block`: flusso di testo, ma con proprietà di blocco
- `flex`, `grid`, `table`, `none`

Flexbox e Grid

Flexbox: layout monodimensionale (asse principale e trasversale).

```css
display: flex
flex-direction: row | row-reverse | column | column-reverse
justify-content: allineamento sull'asse principale
align-items: allineamento sull'asse trasversale
```

```
flex-wrap: nowrap | wrap | wrap-reverse
flex-grow, flex-shrink, flex-basis
```

Grid: layout bidimensionale (righe e colonne).

```css
display: grid
grid-template-columns, grid-template-rows
grid-gap (o gap)
grid-column, grid-row
```

Confronto: Flexbox facilita distribuzione e allineamento lungo un asse; Grid facilita il controllo simultaneo di righe e colonne.

Media query e strategie responsive

Media query: regola condizionale che applica CSS quando caratteristiche del dispositivo/viewport rispettano una condizione.

```css
@media (max-width: 700px) { ... }
```

Strategie:

- **Fixed layout**: larghezza fissa
- **Fluid layout**: dimensioni relative
- **Adaptive**: layout distinti per intervalli di larghezza
- **Responsive**: approccio che combina griglie fluide, immagini flessibili e media query

Il meta viewport evita che i browser mobili trattino la pagina come un grande viewport virtuale:

```html
<meta name="viewport" content="width=device-width, initial-scale=1">
```

Da ricordare. *Mobile first*: progettare per dispositivi mobili e intervenire con media query per schermi più grandi.

7. CSS moderno, adattivo e accessibile

Pseudo-classi funzionali:

- `:is(A,B)`: seleziona chi soddisfa almeno uno tra i selettori
- `:where(A,B)`: come `:is`, ma con **specificità zero**
- `:not(A)`: esclude
- `:has(A)`: seleziona un elemento in base a elementi correlati

Custom properties (variabili CSS):

```css
:root {
  --brand: #245b87;
  --space: 1rem;
}
.button {
  color: var(--brand);
  padding: var(--space);
}
.box {
```

```
10    width: min(100%, 60rem);
11    font-size: clamp(1rem, 2vw, 1.5rem);
12  }
```

Funzioni di calcolo:

- `calc()`
- `min()`
- `max()`
- `clamp(min, preferito, max)`

Container queries (`@container`): scelgono lo stile in funzione delle dimensioni del contenitore.

Immagini responsive:

- `srcset` per densità o larghezza
- `<picture>` per art direction

Preferenze e feature detection:

- `@media (prefers-reduced-motion: reduce)`
- `@media (prefers-color-scheme: dark)`
- `@supports (display: grid)`

Nesting CSS: consente di annidare selettori.

Da ricordare. `@media` guarda il viewport, `@container` il contenitore; `srcset` è selezione della risorsa adatta, `<picture>` può cambiare la direzione artistica; `:where()` è volutamente a specificità zero.

Parte V — JavaScript

8. JavaScript: linguaggio, scope, funzioni e oggetti

JavaScript: linguaggio di programmazione general-purpose, dinamicamente e debolmente tipizzato, specificato da **ECMAScript**.

Inclusione, sintassi e operatori

Il codice può essere incorporato con `<script>...</script>` o importato con `<script src="..."></script>`.

`defer` scarica in parallelo ed esegue quando il parsing è completo, rispettando l'ordine. `async` esegue appena possibile senza garantire l'ordine. Gli script `type="module"` sono moduli e hanno comportamento di caricamento differito per impostazione predefinita.

Strict mode (`'use strict';`) abilita vincoli che rendono errori pratiche storicamente tollerate. I moduli e il corpo delle classi sono già strict.

Tipi primitivi: `number`, `bigint`, `string`, `boolean`, `undefined`, `symbol`, `null`.

Operatori: aritmetici (`+`, `-`, `*`, `/`, `%`, `**`), confronti (`<`, `<=`, `>`, `>=`), logici (`&&`, `||`, `!`), `??` (nullish coalescing), ternario (`cond ? a : b`).

Uguaglianza: `==` (debole, con conversioni), `===` (stretta, preferita).

Variabili, hoisting e scope

- `let`: variabile riassegnabile con **scope di blocco**
- `const`: binding non riassegnabile, **scope di blocco** (non rende immutabile l'oggetto referenziato)
- `var`: scope di funzione, sconsigliata nel codice moderno

Scope lessicale: la visibilità di una variabile dipende da dove il codice è scritto.

Hoisting: il linguaggio elabora le dichiarazioni prima di eseguire.

- Funzione dichiarata con `function f(){...}` può essere chiamata prima della sua dichiarazione
- `var` risulta inizialmente `undefined`
- `let`/`const` esistono ma non sono utilizzabili prima dell'inizializzazione (**Temporal Dead Zone - TDZ**), pena `ReferenceError`

Funzioni, espressioni e closure

- **Function declaration**: `function sum(a,b){return a+b;}`
- **Function expression**: `const sum = function(a,b){...}`
- **Arrow function**: `const sum = (a,b) => a+b`

Le funzioni sono **first-class values**: possono essere assegnate, passate e restituite.

I parametri possono avere default; `...args` (rest parameter) raccoglie argomenti; `...arr` (spread) espande un iterabile.

Closure: combinazione di una funzione e del suo ambiente lessicale, che permette alla funzione di accedere a variabili esterne anche dopo la conclusione della funzione che l'ha creata.

Oggetti, riferimenti, metodi, this

Oggetto: collezione dinamica di proprietà chiave–valore; le chiavi sono stringhe o simboli.

Si crea con un **object literal**:

```JavaScript
{ name: 'Ada', age: 24 }
```

Le proprietà si leggono/scrivono con `obj.name` e `obj['name']`; il secondo accetta una chiave calcolata.

`delete obj.age` rimuove una proprietà configurabile; `in` verifica la presenza lungo la catena dei prototipi, `Object.hasOwn(obj,key)` solo quella propria.

Gli oggetti sono assegnati **per riferimento**: `let b = a` non clona l'oggetto. Il confronto `a === b` verifica l'identità del riferimento.

Shallow copy (`{...a}`, `Object.assign`) copia il livello superficiale: oggetti annidati rimangono condivisi. **Deep copy** richiede una copia ricorsiva o strumenti adeguati (es. `structuredClone`).

Un **metodo** è una funzione memorizzata in una proprietà. In una chiamata `obj.method()`, il valore di `this` dipende dal modo in cui viene invocata la funzione.

Le **arrow function** non hanno `this` proprio e lo catturano lessicalmente.

`call`, `apply` e `bind` permettono di gestire il contesto di chiamata.

Le **funzioni costruttrici** sono invocate con `new`: creano un oggetto collegato al prototipo della funzione, assegnano `this` al nuovo oggetto.

La sintassi `class` è oggi preferibile per descrivere costruttori e metodi.

Optional chaining `user?.address?.city` restituisce `undefined` se un tratto è `null`/`undefined` senza lanciare un errore.

`??` fornisce un fallback senza scartare valori validi come `0`, `false` o stringa vuota.

9. Ereditarietà prototipale, strutture dati, classi, moduli

Prototipi

Ereditarietà prototipale: ogni oggetto ordinario può riferirsi a un altro oggetto tramite il proprio `[[Prototype]]`. Se una proprietà non esiste nell'oggetto, la ricerca prosegue sulla **prototype chain** fino a trovarla o a incontrare `null`.

L'oggetto corrente può **sovrascrivere** una proprietà ereditata senza modificarne necessariamente il valore nel prototipo.

`Object.getPrototypeOf` e `Object.create` permettono di esaminare e creare legami prototipali.

`prototype` della funzione costruttrice e **`[[Prototype]]` di un'istanza** sono concetti diversi: quando si invoca `new F()`, il nuovo oggetto ha normalmente `F.prototype` come prototipo.

Array, destructuring, iterabili, Map e Set

Array: oggetto indicizzato e dinamico con proprietà `length`; può contenere valori eterogenei ed essere annidato.

Metodi frequenti:

- `push`/`pop` (fine)
- `unshift`/`shift` (inizio)
- `slice` (sottosequenza senza mutare)
- `splice` (modifica in place)
- `map` (trasformazione)
- `filter` (selezione)
- `find` (primo elemento)
- `reduce` (accumulazione)
- `forEach` (effetti)

`map` restituisce un nuovo array e `forEach` normalmente no.

`for...of` itera valori; `for...in` itera chiavi enumerabili (non intercambiabili).

Destructuring

```JavaScript
const [a, b] = [1, 2]; // a = 1, b = 2
const {name, age} = person;
```

Map e Set

- **Map**: mappa chiave-valore, chiavi possono essere di qualsiasi tipo
- **Set**: insieme di valori unici

Moduli

```JavaScript
// Esportazione
export const name = 'value';
export function func() {}

// Importazione
import { name, func } from './module.js';
```

Capitolo 5

Parte VI — Architetture e Best Practice

Architetture client-server e multi-tier

Client-server: il client (browser) richiede risorse, il server fornisce risposte.

Server HTTP può essere **statico** (file) o **dinamico** (elaborazione).

Multi-tier: separazione in livelli:

- **Presentation** (interfaccia utente)
- **Application** (logica applicativa)
- **Data** (persistenza)

Single-page application (SPA): applicazione che carica una singola pagina HTML e aggiorna dinamicamente il contenuto tramite JavaScript.

API REST

REST (Representational State Transfer): stile architetturale per sistemi distribuiti.

Principi:

- **Risorse** identificate da URI
- **Rappresentazioni** (HTML, JSON, XML)
- **Metodi HTTP** con semantica standard
- **Stateless**: ogni richiesta contiene tutte le informazioni necessarie

Design REST:

- URI: `/risorsa` (plurale), `/risorsa/{id}`
- Metodi: `GET` (recupero), `POST` (creazione), `PUT` (sostituzione), `PATCH` (modifica parziale), `DELETE` (eliminazione)
- Codici HTTP: 200 OK, 201 Created, 204 No Content, 400 Bad Request, 401 Unauthorized, 403 Forbidden, 404 Not Found, 405 Method Not Allowed

Da ricordare. REST è uno stile, non uno standard; le API RESTful usano i metodi HTTP secondo la loro semantica.

Sicurezza Web

- **XSS (Cross-Site Scripting)**: iniezione di codice malevolo
- **CSRF (Cross-Site Request Forgery)**: richiesta non autorizzata da parte di un utente autenticato
- **SQL Injection**: iniezione di codice SQL
- **CORS (Cross-Origin Resource Sharing)**: meccanismo per controllare l'accesso alle risorse tra origini diverse

Best Practice

- **Semantica**: usare elementi HTML semantici
- **Accessibilità**: garantire che il sito sia accessibile a tutti
- **Performance**: ottimizzare il caricamento delle risorse
- **SEO**: ottimizzare per i motori di ricerca
- **Manutenibilità**: codice pulito, ben strutturato e documentato